THI THUY DUNG NGUYEN
Student #: 221372438

EECS3311 Fall 2026 Course Project
Stage-1 Instruction

1. Description: 2
2. Feature Specifications: 4
3. Design Your System Using UML 13
4. Feature-to-Design Traceability 26
5. Explain How Each Feature Is Realized 28

1. **Description:**
    - AI-Driven Personal Food Discovery & Mapping Platform

    - Problem: Traditional food discovery platforms rely heavily on aggregated public ratings and reviews. However, a restaurant with a high overall rating may not necessarily match an individual's personal taste, dietary preferences, budget, or dining habits. Users also have to search across different applications to find restaurants, save places they want to visit, remember restaurants they have already tried, and share food experiences with friends. This can make food discovery time-consuming and less personal.

    - Target users:
        - Individuals who frequently search for restaurants and want personalized recommendations.
        - Users who want restaurant recommendations based on their own tastes rather than only public ratings.
        - Users who want to keep track of restaurants they have visited, liked, or want to try.
        - Users who want a simple private way to share food experiences with close friends.
        - Users who want to visually organize their food experiences using a personal map.

    - Major Functionality:
        - Restaurant discovery and search.
        - Personalized AI restaurant recommendations.
        - A conversational AI food assistant.
        - Personal restaurant ratings, reviews, notes, and history.
        - Automatic learning of the user's food preferences.
        - A personal interactive food map.
        - Saving restaurants and organizing them into custom collections.
        - Automatic restaurant categorization and tagging.
        - Quick food-photo sharing with friends.
        - A private Locket-style feed showing friends' recent food experiences.
        - Saving restaurants discovered through friends.
        - Personalized refinement based on user feedback.
    - Agent description: The agent accepts natural-language food requests and determines the user's intent, constraints, and preferences. It can retrieve restaurant information, access the user's personal food memory, compare candidate restaurants, rank them, and explain why particular restaurants are suitable.

1. Process the user's request.
2. Identify constraints such as cuisine, location, price, and previous-visit exclusion.
3. Retrieve relevant restaurants.
4. Retrieve the user's saved restaurants, ratings, notes, preferences, and dining history.
5. Filter restaurants that violate the user's constraints.
6. Rank the remaining candidates according to the user's personal taste profile.
7. Explain why the highest-ranked restaurants are suitable.
8. Allow the user to save a restaurant directly to their personal map.
9. Learn from the user's response or later rating.

- An AI agent is appropriate because restaurant recommendations require combining many different types of information. A deterministic search system could filter restaurants by cuisine, price, or location, but it would have difficulty understanding requests expressed naturally and combining them with subjective personal preferences. The AI agent can reason over multiple constraints and personal context simultaneously. It can also learn from feedback over time. For example, if a user repeatedly gives high ratings to quiet Japanese restaurants and low ratings to crowded restaurants, the system can gradually incorporate these preferences into future recommendations.
- AI/LLM model(s):
    - An LLM for natural-language understanding, reasoning, recommendation generation, and conversational interaction.
    - OpenAI embedding models for converting user notes, reviews, preferences, and other food-related memories into vector representations for semantic retrieval.
- Overall architecture: The AI model is accessed through the LLMEngine interface, which allows the system to use different language model providers through LLMFactory and adapters. When a user submits a request, the Agent extracts constraints, retrieves relevant memories and restaurant candidates, and uses the AI model to generate recommendations and explanations. The results are then filtered and ranked using user preferences and ranking strategies before being presented to the user. User feedback can update the taste profile and food memory, helping improve future recommendations.

2. **Feature Specifications:**

a. F01: Personal Food Map Visualization

Description:	The main application view is an interactive personal food map that displays the user's saved, visited, and AI-recommended restaurants. Users can explore restaurants geographically, filter locations by category or collection, select restaurant pins to view details, and save new restaurants directly from the map.
User Interaction:	The user opens the application and is immediately shown their personal food map. They can: ● Select restaurant pins. ● Zoom and move around the map. ● Filter restaurants. ● View saved or visited places. ● Open restaurant details. ● Ask the AI agent to find restaurants and display them on the map.
Input:	● User location or selected map area. ● Restaurant data. ● Saved restaurants. ● Visited restaurants. ● User collections. ● AI recommendation results. ● Map filters.
Output:	● Interactive map. ● Restaurant pins. ● Restaurant information cards. ● Filtered map layers. ● AI-recommended restaurant locations.
AI Involvement:	Hybrid Map display and filtering are deterministic, while AI-generated recommendations can appear as recommended locations.
Expected workflow:	● User opens the application. ● The system loads the user's saved and visited restaurants. ● Restaurant locations are displayed on the map. ● Recommended restaurants are highlighted on the map.

	• The user selects a restaurant pin to view its details.
Error/Alternative Cases:	• If a restaurant has no valid location, it is displayed without a map pin. • If no restaurants match the selected filters, the system displays an empty-state message. • Map service unavailable → display restaurant information in a list instead.

b. F02: Personal Reviewing, Logging

Description:	Allows users to record their personal experiences with restaurants through ratings, reviews, notes, photos, tags, and saved status.
User Interaction:	The user opens a restaurant card and can rate it, write a note, add tags/photos, mark it as visited, or save it for later.
Input:	• Restaurant. • Star rating. • Written review/note. • Photo. • Tags. • Visited/saved status.
Output:	A stored personal restaurant record.
AI Involvement	Hybrid. Storing the information is deterministic, while the information becomes part of the user's AI memory.
Expected Workflow:	• User selects a restaurant. • User enters a rating, note, or other information. • The system validates the input. • The restaurant record is stored. • The information is added to the user's personal food memory. • Future recommendations can use this information.
Error/Alternative Cases:	• Invalid rating → reject the value and request a valid rating.

	• Restaurant already exists → update the existing record rather than creating an unnecessary duplicate. • Photo upload fails → save the review without the photo.

c. F03: Conversational AI Recommendation Assistant
   - Understand what the user wants → gather relevant restaurant information → consider the user's personal taste → recommend → explain → let the user take action.

Description:	Provides a conversational interface where users can naturally ask the AI what they should eat or where they should go.
User Interaction:	The user enters a natural-language request through the AI chat interface.
Input:	• Cuisine. • Location. • Budget. • Dietary restrictions. • Time. • Dining occasion. • Personal preferences. • Previous restaurant experiences.
Output:	• Ranked restaurant recommendations. • Restaurant cards. • Short explanations. • Recommended restaurants highlighted on the map.
AI Involvement:	AI-based.
Expected workflow:	• User enters a natural-language request. • Agent identifies constraints and preferences. • Agent retrieves relevant restaurant information. • Agent retrieves relevant personal food memory. • Candidate restaurants are filtered. • Candidates are ranked according to the user's preferences. • The LLM generates explanations for the recommendations.

	• Results are displayed as restaurant cards and shown on map.
Error/Alternative Cases:	• No restaurant satisfies all constraints → relax lower-priority constraints and explain the change. • Restaurant information unavailable → exclude it and continue with available candidates. • Request is ambiguous → ask the user for the missing information.

d. F04: Personalized Taste Profile

Description:	Automatically builds and updates a personal food preference profile based on the user's ratings, reviews, notes, saved restaurants, visits, and feedback.
User Interaction:	The user can view their current taste profile and continue using the application normally while the system updates the profile in the background.
Input:	• Restaurant ratings. • Reviews and notes. • Saved restaurants. • Visited restaurants. • Food categories. • Explicit preferences. • AI recommendation feedback.
Output:	A structured personal taste profile containing preferences such as cuisines, price ranges, restaurant characteristics, and other learned preferences.
AI Involvement:	AI-based.
Expected workflow:	• User interacts with restaurants through the application. • Relevant information is added to personal memory. • The system analyzes explicit and implicit preferences. • The AI updates the user's preference profile. • Future recommendations use the updated profile.

Error/Alternative Cases:	● Not enough user history → use explicit preferences and general restaurant information. ● Conflicting preferences → prioritize recent or stronger evidence. ● User changes a preference → update the profile accordingly.

e. F05: Quick Photo Experience Sharing

Description:	Allows users to quickly share a instant food photo in real time and optionally attach a restaurant, short note, or reaction to document their food experience.
User Interaction:	The user takes a food photo in real time and can optionally select the associated restaurant, add a short caption, and publish the post to their friends.
Input:	Food photo. Restaurant. Optional caption or note. User's posting action.
Output:	An experience they want to share to their friends right away what happen
AI Involvement:	Hybrid Photo posting and feed publishing are deterministic, while AI can use the restaurant and experience information as part of the user's food memory and future recommendations.
Expected workflow:	● User goes to the photo-sharing. ● User takes a photo, can be about the food, restaurant, places, atmosphere, etc. ● The system allows the user to select the associated restaurant. ● User optionally adds a caption or note. ● The post is published to the user's friends. ● The restaurant and experience can be added to the user's personal food history.
Error/Alternative Cases:	● Photo upload fails → display an error and allow the user to retry. ● Restaurant cannot be identified → allow the user to publish the photo without a restaurant.

	• User cancels before posting → discard the unfinished post.

f. F06: Friends' Activity Feed
   - Friends → private food feed → photo + restaurant → reaction → Save to My Map

Description:	Displays a private stream of close friends' recent dining photos, notes, and visited restaurants using a simple Locket-style interaction model.
User Interaction:	The user opens the Friends Feed and views recent food posts. Selecting a post opens the associated restaurant, where the user can save it to their own map.
Input:	• Friends list. • Friends' food posts. • Restaurant information.
Output:	A private chronological food feed.
AI Involvement:	Hybrid Feed generation is deterministic, while restaurant information from the feed can be incorporated into personalized discovery.
Expected workflow:	• Friend creates a food post. • The post is added to the private feed. • User opens the Friends Feed. • User views the photo and restaurant. • User can open the restaurant. • User can save the restaurant to their own map.
Error/Alternative Cases:	• No recent posts → display an empty-feed message. • Restaurant information unavailable → show the photo and available information. • Friend removes a post → remove it from the feed.

g. F07: Restaurant Discovery & Search

Description:	Allows users to browse and search for restaurants using available restaurant information, location, cuisine, price range, and other basic filters. General, basic feature
User Interaction:	The user searches for restaurants or browses available restaurants in a selected area. They can apply filters and select a restaurant to view its details.
Input:	● Search query. ● User location or selected area. ● Cuisine. ● Price range. ● Restaurant categories. ● Other search filters.
Output:	A list of matching restaurants with basic information such as name, location, cuisine, price range, and rating. Restaurant results can also be displayed on the personal map.
AI Involvement:	Deterministic Basic restaurant search and filtering use structured restaurant data. The AI recommendation assistant can use this functionality to retrieve candidate restaurants.
Expected workflow:	● User opens the restaurant discovery interface. ● The system retrieves restaurants from the available restaurant data source. ● User enters a search query or selects filters. ● The system filters and ranks matching restaurants. ● Restaurant results are displayed as cards and/or map locations. ● User can open a restaurant to view its details, save it, or record a personal experience.
Error/Alternative Cases:	● No matching restaurants → display an empty-state message. ● Restaurant information unavailable → exclude incomplete results or display the available information. ● Search service unavailable → display an error and allow the user to retry.

h. F08: Restaurant Recommendation History

Description:	Stores previous AI-generated restaurant recommendations so users can revisit restaurants they previously considered, even after the original AI conversation has ended.
User Interaction:	The user opens their recommendation history and browses previously recommended restaurants. They can open a restaurant, save it, add it to a collection, or remove it from their history.
Input:	AI-generated recommendations, recommendation date/time, recommendation context, user actions.
Output:	A chronological list of previously recommended restaurants.
AI Involvement:	Hybrid The AI generates the original recommendations, while storing and retrieving recommendation history is deterministic.
Expected workflow:	● User asks the AI for recommendations. ● AI generates restaurant recommendations. ● The system automatically stores the recommendations. ● User later opens Recommendation History. ● User can revisit the restaurant, save it, review it, or add it to a collection.
Error/Alternative Cases:	● No previous recommendations → display an empty-state message. ● Restaurant is no longer available → indicate that it is unavailable. ● User removes a recommendation → remove it from recommendation history.

i. F09: Automatic Categories Tagging

Description:	Automatically assigns useful food-related categories or tags to restaurants and user food records.
User Interaction:	Users see automatically generated tags on restaurant cards and can manually correct or remove them.

Input:	• Restaurant information. • Menu/category information. • User notes and reviews. • Restaurant descriptions.
Output:	A set of restaurant categories/tags.
AI Involvement:	AI-based / Hybrid The AI suggests semantic categories, while the user can manually modify them.
Expected workflow:	• Restaurant information is retrieved. • The AI analyzes the available information. • Relevant categories are generated. • Tags are attached to the restaurant. • User can modify the generated tags.
Error/Alternative Cases:	• Insufficient information → generate fewer tags. • Incorrect tag → user can remove or correct it. • Conflicting categories → retain multiple relevant categories rather than forcing one.

j. F10: Save & Custom Food Collections

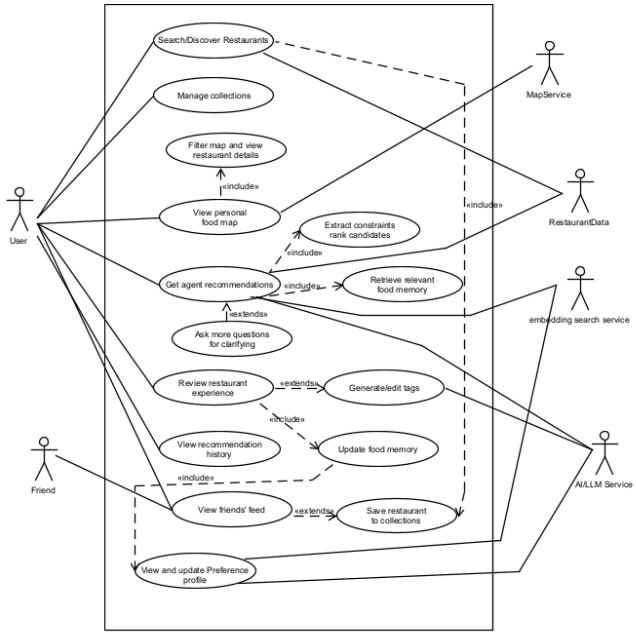
Description:	Save places into collections/lists. Organizes saved restaurants into custom themed lists (e.g., "Date Night") and toggles list layers on map.
User Interaction:	The user creates a collection and adds or removes saved restaurants from it. Collections can also be displayed as layers on the personal map.
Input:	• Collection name. • Saved restaurants. • Optional description/category.
Output:	A user-defined restaurant collection.
AI Involvement:	Deterministic / Hybrid Collection management is deterministic, while the AI may use collection information when making recommendations.
Expected workflow:	• User creates a collection. • User selects saved restaurants. • Restaurants are added to the collection. • Collection appears in the user's map/list.

	• User can enable or disable the collection as a map layer.
Error/Alternative Cases:	• Duplicate restaurant → do not add it twice. • Empty collection → allow it but indicate that it contains no restaurants. • Collection deleted → restaurants remain saved but are removed from that collection.

3. Design Your System Using UML

a. Class Diagram

- Diagram: UML Diagrams



- Facade:
  - Problem: a recommendation requires multiple operations: understanding request, retrieving memories, searching restaurants, filtering, ranking, and explaining results
  - Participants: FoodAgent is the facade; ConstraintExtractor, CandidateFilter, RestaurantService, MemoryService, RankingStrategy, and LLMEngine are the subsystems
  - The rest of the application can call processRequest() without knowing the entire AI workflow
  - The user would need to coordinate many components directly, making changes harder without it

- Strategy:
    - Problem: different user and citations require different ways of ranking restaurants
    - Participants: RankingStrategy defines the interface; TasteBasedRanking and DistanceBasedRanking implement different algorithms; FoodAgent selects and uses a strategy
    - Then we can prioritize personal preferences, distance, or even a price-based strategy without changing much
    - Ranking logic would likely accumulate in large conditional statements inside FoodAgent without it
- Factory Method
    - Problem: The application needs to create an appropriate LLM implementation, such as a local model or a cloud-based model
    - Participants: LLMEngine is the product interface; LocalLLMEngine and OpenAILLMEngine are concrete products; LLMFactory creates the chosen implementation
    - The application can select its LLM based on configuration instead of constructing provider-specific classes throughout the code
    - Without this design, model creation and provider-selection logic would be duplicated across the application, making it harder to switch providers
- Observer:
    - Problem: When a user likes or dislikes a recommendation, several components may need to react
    - Participants: FeedbackPublisher manages subscriptions and sends notifications; FeedbackObserver defines the notification interface; TasteProfileUpdater updates preferences; HistoryRecorder records recommendation history
    - The feedback system can notify multiple independent components without directly depending on their concrete implementations
    - Without it, FoodAgent would need to call every feedback-related component individually. Adding another reaction, such as updating analytics, would require changing the agent
- Adapter:
    - Problem: External AI and map services may have different method names, input formats, and response formats
    - Participants: Each adapter implements the application's expected interface (LLMEngine or MapProvider) and translates calls to an external client's API

- The rest of the system can work with consistent interfaces even when the external service has a different API
- Without it, provider-specific API details would spread into FoodAgent and MapService, making integrations and provider changes more difficult

b. Use-Case Diagram

i. UC01: View personal map

- Actors: User, Map Service, Restaurant data service
- Allowing users to visualize and explore their saved, visited, recommended restaurants on an interactive map
- Preconditions:
  - The user has opened the application
  - Restaurant records and the user's saved data are available
- Trigger: the user open the main application view
- Main success scenario:
  - The system retrieves the user's saved and visited restaurants
  - The system retrieves their locations and any available recommendation results
  - The Map Service displays the map and restaurant pins
  - The user zooms, moves around the map, or applies filters
  - The user selects a pin to view restaurant details
  - The user can save a restaurant or request AI recommendations
- Alternative/Exception Flows:
  - If a restaurant has no valid location, its information is displayed without a map pin
  - If no restaurants match the filters, the system displays an empty-state message
  - If the Map Service is unavailable, the system displays restaurants in a list
- Postconditions: The user can view and explore their personal restaurant map
- Related Feature(s): F01, F07, F10

ii. UC02: Restaurant Experience Log

- Actors: User, AI / LLM Service
- Goal: Allow users to record their restaurant experiences and preserve them for future reference and personalized recommendations

- Preconditions:
    - The user has opened a restaurant's details
    - The restaurant record is available
- Trigger: The user chooses to add or edit a restaurant experience
- Main Success Scenario:
    - The user enters a rating, review, note, photo, or tags
    - The user optionally marks the restaurant as visited or saved
    - The system validates the submitted information
    - The system creates or updates the user's personal restaurant record
    - The system stores the experience in personal food memory
    - The information becomes available to future recommendations
- Alternative/Exception Flows:
    - An invalid rating is rejected, and the user is asked to correct
    - If the user has visited the restaurant before, the system creates a new food log for the new experience rather than overwriting the previous log
    - If the user edits an existing log, the system updates that specific log
    - If the user saves a restaurant without recording a visit, the system updates the saved status rather than creating a new visit log
    - If a photo upload fails, the system saves the other information and allows the user to retry the upload
- Postconditions: The restaurant experience is stored and available for future retrieval
- Related Feature(s): F02, F04, F09

iii. UC03: Conversational AI Recommendation Assistant

- Actors: User, Restaurant Data Service, AI / LLM Service, Embedding Search Service
- Goal: Recommend suitable restaurants based on the user's request, constraints, and personal food preferences
- Preconditions:
    - The user can access the AI assistant
    - Restaurant data and the AI service are available
    - Personal food memory may be available but is not required
- Trigger: The user submits a natural-language request, such as asking where to eat tonight

- Main Success Scenario:
    - The user enters a request describing their needs
    - The agent extracts constraints such as cuisine, location, budget, dietary requirements, and occasion
    - The agent retrieves relevant restaurant information and personal food memories
    - The system filters restaurants that violate the user's constraints
    - The ranking strategy ranks the remaining candidates according to personal preferences
    - The AI generates explanations for the recommended restaurants
    - The system displays ranked restaurant cards and highlights their locations on the map
    - The system automatically records the recommendations in recommendation history
- Alternative/Exception Flows:
    - If the request is ambiguous, the agent asks a clarifying question before continuing
    - If no suitable restaurants are found, the agent relaxes lower-priority constraints and explains the changes
    - Essential constraints, such as allergies and explicit exclusions, are not relaxed
    - If restaurant information is unavailable, the affected candidate is excluded
    - If the AI service is unavailable, the system displays an error and offers basic restaurant search where possible
- Postconditions: The user receives recommendations with explanations, and the recommendation history is updated when results are successfully generated
- Related Feature(s): F01, F03, F04, F07, F08

iv. UC04: Personalized Preference Profile

- Actors: User, AI / LLM Service, Embedding / Vector Search Service
- Goal: Maintain a personal preference profile that improves future restaurant recommendations
- Preconditions:
    - The user has a profile

        - Some preference information may be available from explicit settings or previous interactions
    - Trigger: The user opens their profile, changes a preference, or generates new information through restaurant interactions and recommendation feedback
    - Main Success Scenario:
        - The system collects relevant ratings, reviews, notes, saved restaurants, visits, and feedback
        - The system retrieves relevant personal food memories
        - The AI analyzes the available evidence to identify preferences
        - The system updates the preference profile with relevant cuisines, price ranges, and restaurant characteristics
        - The user views the profile or modifies explicit preferences.
        - Future recommendations use the updated profile
    - Alternative/Exception Flows:
        - If there is insufficient history, the system uses explicit preferences and available restaurant information
        - If preferences conflict, the system considers the strength and recency of the evidence
        - If the user changes a preference, the explicit setting is updated and used in future recommendations
        - If the AI service is unavailable, the system preserves existing preferences and retries the profile analysis later
    - Postconditions: The preference profile is updated when sufficient information is available and can influence subsequent recommendations.
    - Related Feature(s): F02, F03, F04, F05, F06

v. UC-05: Share Food Photo
    - Actors: User, Friend, AI / LLM Service
    - Goal: Allow users to quickly publish food photos and optional restaurant experiences to their friends
    - Preconditions:
        - The user is authenticated.
        - The user has permission to upload and publish content
    - Trigger: The user opens the photo-sharing feature and takes or selects a photo
    - Main Success Scenario:
        - The user takes or selects a food or restaurant photo

        - The user optionally associates the photo with a restaurant
        - The user adds an optional caption or note
        - The user chooses to publish the post
        - The system validates the content and uploads the photo
        - The system publishes the post to the user's permitted friends
        - The restaurant and experience information can be added to personal food memory
    - Alternative/Exception Flows:
        - If photo upload fails, the system displays an error and allows the user to retry
        - If the restaurant cannot be identified, the user can publish without a restaurant association
        - If the user cancels before publishing, the unfinished post is discarded
        - If publication fails, the system informs the user and does not indicate that the post was successfully shared
    - Postconditions: The published photo is visible to authorized friends, and associated experience information is available for future discovery when applicable
    - Related Feature(s): F05, F06, F04

vi. UC-06: View Friends' Activity Feed
    - Actor(s): User, Friend
    - Goal: Allow users to view friends' recent food photos, notes, and restaurant experiences and save interesting restaurants to their own map
    - Preconditions:
        - The user is authenticated
        - The user has friends or permitted connections in the system.
    - Trigger: The user opens the Friends Feed
    - Main Success Scenario:
        - The system retrieves recent published posts from the user's friends
        - The system checks that the user is authorized to view each post
        - The system displays the posts in chronological order
        - The user selects a post to view its photo, caption, and associated restaurant

        - The user opens the restaurant details
        - The user optionally saves the restaurant to their own map or collection
    - Alternative/Exception Flows:
        - If there are no recent posts, the system displays an empty-feed message
        - If restaurant information is unavailable, the system displays the photo and other available post information
        - If a friend removes a post, the post is removed from the feed
        - If the user cannot access a post, the system does not display its private content
    - Postconditions: The user has viewed their accessible friends' posts and may have saved a restaurant to their personal map or collection
    - Related Feature(s): F05, F06, F01, F10

vii. UC-07: Search and Discover Restaurants
    - Actors: User, Restaurant Data Service
    - Goal: Allow users to find restaurants using basic search criteria and structured restaurant information
    - Preconditions:
        - The restaurant search functionality is available
        - Restaurant data is accessible
    - Trigger: The user enters a search query or opens restaurant discovery
    - Main Success Scenario:
        - The user enters a query or selects a location
        - The user optionally applies filters such as cuisine, price range, or category
        - The system retrieves matching restaurant records
        - The system filters and orders the results according to the search criteria
        - The system displays restaurant cards and optionally map locations
        - The user opens a restaurant, saves it, or records a personal experience
    - Alternative/Exception Flows:
        - If no restaurants match, the system displays an empty-state message

        - If some restaurant information is incomplete, the system displays available details or excludes unsuitable results
        - If the search service is unavailable, the system displays an error and allows the user to retry
    - Postconditions: Matching restaurants are displayed, and the user can select a restaurant for further actions
    - Related Feature(s): F01, F03, F07, F10

viii. UC-08: View Recommendation History
    - Actor: User
    - Goal: Allow users to revisit previously recommended restaurants and manage their recommendation history
    - Preconditions:
        - The user is authenticated
        - Recommendation history may contain previously generated results
    - Trigger: The user opens Recommendation History.
    - Main Success Scenario:
        - The system retrieves the user's saved recommendation history
        - The system displays recommendations chronologically, including their dates and available context
        - The user selects a previous recommendation
        - The system displays the associated restaurant details
        - The user can save the restaurant, add it to a collection, record a review, or remove the recommendation from history
    - Alternative/Exception Flows:
        - If no history exists, the system displays an empty-state message
        - If a restaurant is no longer available, the system indicates its unavailable status when known
        - If the user removes a recommendation, it is removed from their history without necessarily deleting the restaurant or other users' records
    - Postconditions: The user has reviewed their recommendation history and any requested history changes are saved
    - Related Feature(s): F03, F08, F01, F02, F10

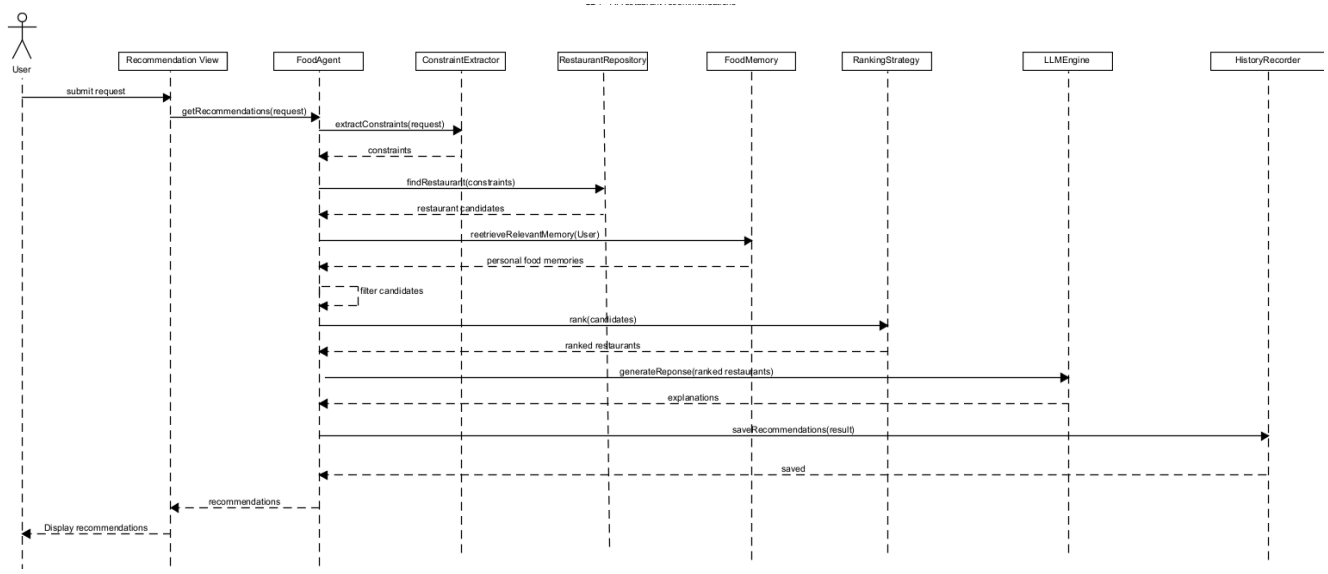
ix. UC-09: Generate and Edit Automatic Tags
- Actors: User, AI / LLM Service
- Goal: Generate useful categories and tags for restaurants and personal food records, while allowing users to correct inaccurate tags
- Preconditions:
  - Restaurant information, a review, or another food record is available
  - The AI service is available for automatic tag generation
- Trigger: A restaurant or food record is created, updated, or submitted for tagging
- Main Success Scenario:
  - The system retrieves available restaurant descriptions, menu information, reviews, or notes
  - The AI analyzes the available information
  - The AI suggests relevant food categories and tags
  - The system associates the suggested tags with the appropriate record
  - The user views the tags and optionally edits or removes them
  - The system saves the final tags
- Alternative/Exception Flows:
  - If information is insufficient, the system generates fewer tags or leaves the record untagged
  - If a tag is incorrect, the user can remove or replace it
  - If AI tag generation fails, the record can still be saved and tagged later
- Postconditions: The record contains the available tags, including any user corrections
- Related Feature(s): F02, F04, F09, F10

x. UC-10: Manage Food Collections
- Actor: User
- Goal: Allow users to organize restaurants into custom collections and display those collections on their personal map
- Preconditions:
  - The user is authenticated
  - The restaurant records to be added are available
- Trigger: The user creates a collection or chooses to manage an existing collection

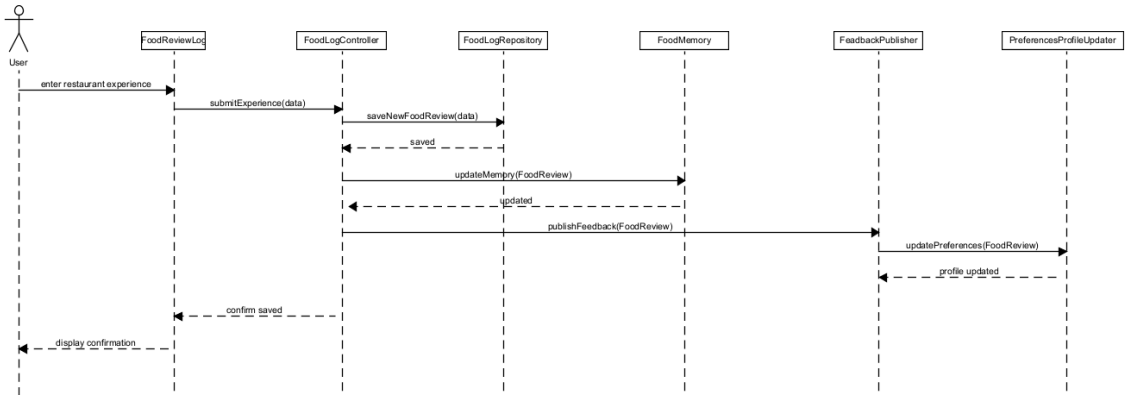
- Main Success Scenario:
    - The user creates a collection and enters its name and optional description
    - The user selects restaurants to add
    - The system checks for duplicate entries
    - The system saves the collection and its restaurant associations
    - The user views, edits, or removes restaurants from the collection
    - The user optionally enables the collection as a map layer
- Alternative/Exception Flows:
    - If a restaurant is already in the collection, the system does not add it twice
    - If a collection is empty, the system allows it and indicates that it contains no restaurants
    - If a collection is deleted, its restaurant records remain saved independently, but their associations with that collection are removed
- Postconditions: The collection and its restaurant associations reflect the user's changes
- Related Feature(s): F01, F07, F10, F06

c. Sequence Diagrams

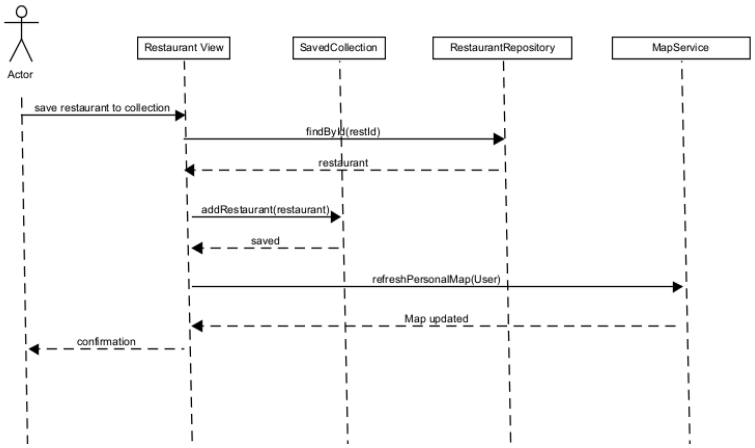
i. SD1: AI restaurant recommendations



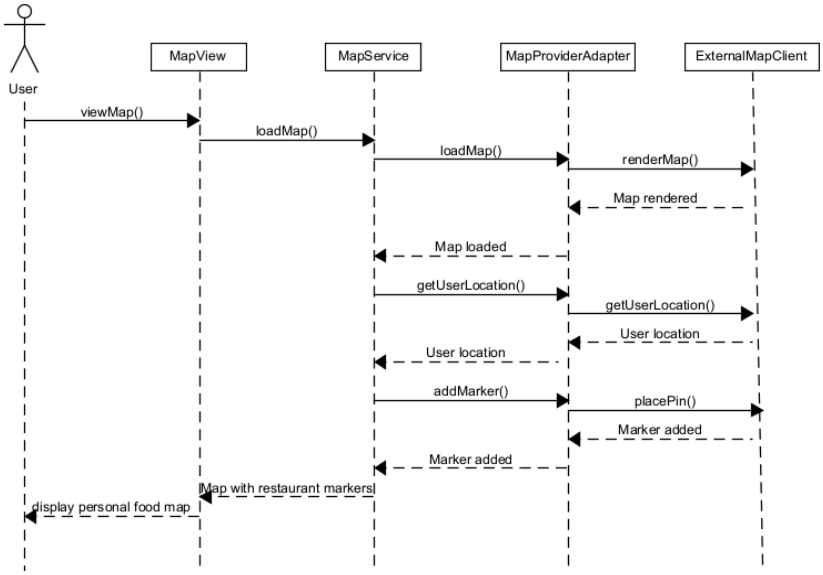
ii. SD02: Create Food Review



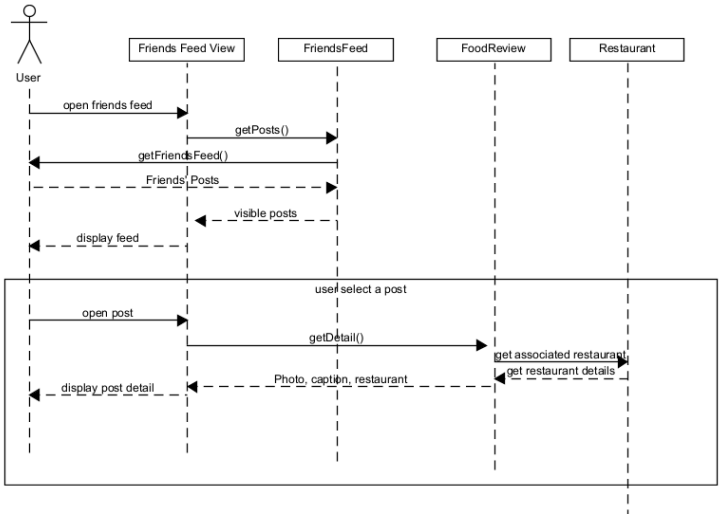
iii. SD3: Save a restaurant to a collection



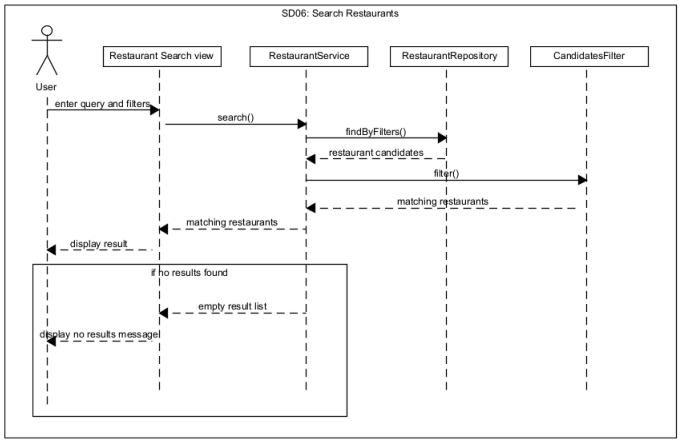
iv. SD04: Load Personal Food Map



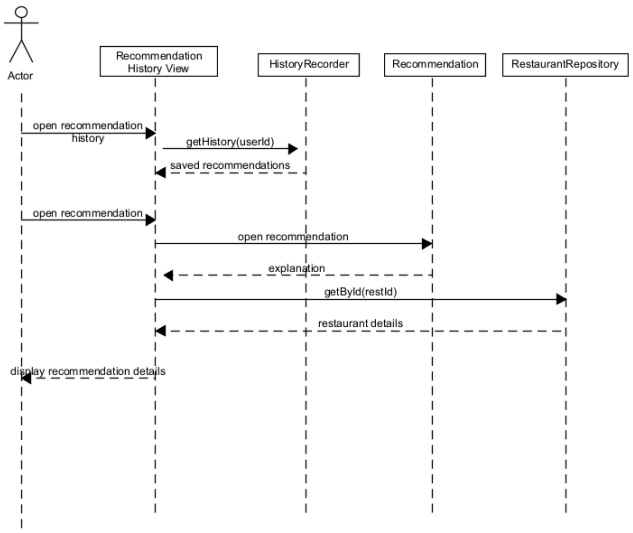
v. SD05: View Friends' Activity Feed



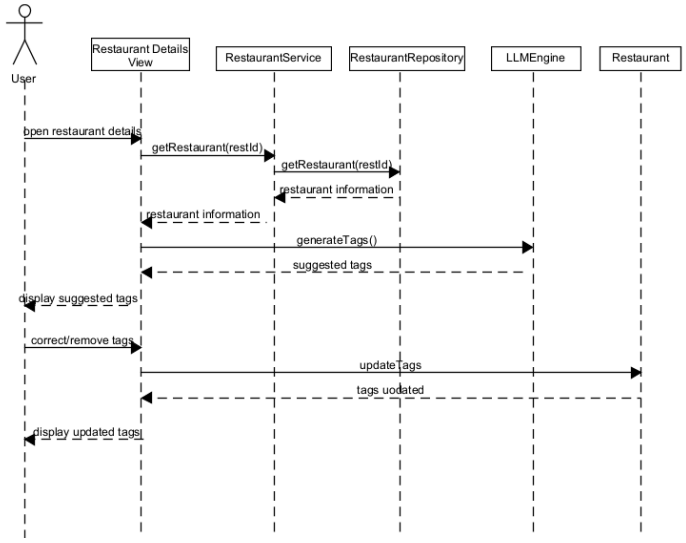
vi. SD06: Search Restaurants



vii. SD07: View Recommendation History



viii. SD08: Generate Automatic Categories and Tags



4. Feature-to-Design Traceability

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Personal Food Map	Hybrid	UC01: View Personal Map	MapService, MapProvider, MapProviderAdapter, ExternalMapClient, RestaurantRepository, Restaurant	loadMap(), addMarkers(), findByLocation()	SD04: Load Personal Food Map	Adapter
F02	Restaurant Experience Log	Hybrid	UC02: Restaurant Experience Log	FoodReview, User, MemoryService, FoodMemory, Restaurant	publish(), saveMemory(), createReview(), editReview	SD02: Create Food Review	Observer
F03	Conversational AI Recommendation Assistant	AI-based	UC03: Conversational AI Recommendation Assistant	Food Agent, RecommendationRequest, ConstraintExtractor, CandidatesFilter, RestaurantService, MemoryService, RankingStrategy, LLMEngine, Recommendation	processRequest(), extractConstraints(), retrieveRelevantMemory(), search(), filter(), rankRestaurants(), reason()	SD01: Get AI Recommendation	Facade, Strategy, Factory Method, Adapter

F04	Personalized Taste Profile	AI-based	UC04: Update Taste Profile	User, UserPreferences, FoodMemory, MemoryService, FeedBackPublisher, TasteProfileUpdater	retrieveRelevantMemory(), saveMemory(), updateMemory(), onFeedBack(), updatePreference()	SD02: Create Food Review	Observer
F05	Quick Photo Experience Sharing	Hybrid	UC05: Share Food Photo	FoodReview, User, FriendsFeed, MemoryService	publish(), saveMemory(), getPosts()	SD02: Create Food Review	
F06	Friends' Activity Feed	Hybrid	UC06: View Friends' Feed	User, FriendsFeed, FoodReview, Restaurant	getFriendsFeed(), getPosts(), getPostByFriends()	SD05: View Friends' Activity Feed	
F07	Restaurant Discovery & Search	Deterministic	UC07: Search Restaurants	RestaurantService, RestaurantRepository, Restaurant, MapService	search(), findByFilters(), findByLocation(), getById(	SD06: Search Restaurants	Adapter
F08	Restaurant Recommendation History	Hybrid	UC08: View Recommendation History	Food Agent, Recommendation, HistoryRecorder, FoodReview	getRecommendations(), recordRecommendations()	SD07: View Recommendation History	
F09	Automatic Categories Tagging	AI-based/Hybrid	UC09: Generate and Edit Automatic Tags	ConstraintExtractor, LLMEngine, FoodReview, Restaurant, Tag	generateTags(), getPhotos(), getDetail()	SD08: Generate Automatic Categories and Tags	
F10	Save & Custom Food Collections	Deterministic /Hybrid	UC10: Manage Food Collections	User, Collections, SavedCollection, CustomCollection, Restaurant, MapService	saveRestaurant(), addRestaurant(), removeRestaurant(), getRestaurants(), refresh()	SD3: Save a restaurant to a collection	

5. **Explain How Each Feature Is Realized**
    - F01: Personal Food Map Visualization
        - Related Use Case: UC01: View Personal Map
        - Related Sequence Diagram: SD04: Load Personal Food Map
        - Classes involved:
            - MapService — manages map loading and restaurant markers
            - MapProvider — defines the map interface
            - MapProviderAdapter — connects the application to the external map service
            - RestaurantRepository — retrieves restaurant records
        - Important methods:
            - MapService.loadMap()
            - MapService.addMarkers()
            - RestaurantRepository.findByLocation()
        - Execution: When the user opens the map, MapService.loadMap() loads the map through MapProviderAdapter. The system retrieves restaurant locations and uses addMarkers() to display pins. The Adapter pattern allows the application to communicate with the external map service through a consistent interface.
    - F02: Personal Reviewing and Logging
        - Related Use Case: UC02: Restaurant Experience Log
        - Related Sequence Diagram: SD02: Create Food Review
        - Classes involved:
            - FoodReview — stores ratings, photos, captions, and review information
            - User — initiates review creation
            - FoodMemory — stores personal restaurant experiences
            - MemoryService — manages memory storage
        - Important methods:
            - FoodReview.publish()
            - FoodMemory.addReview()
            - MemoryService.saveMemory()
        - Execution: When the user submits a restaurant review, the system creates a FoodReview and stores the experience in FoodMemory. MemoryService.saveMemory() preserves relevant information for future recommendations.
    - F03: Conversational AI Recommendation Assistant
        - Related Use Case: UC03 — Conversational AI Recommendation Assistant

    - Related Sequence Diagram: SD01 — Generate AI Restaurant Recommendations
    - Classes involved:
        - Food Agent — coordinates the recommendation process
        - ConstraintExtractor — identifies user requirements
        - RestaurantService — retrieves restaurant candidates
        - MemoryService — retrieves personal food memories
        - RankingStrategy — ranks candidate restaurants
        - LLMEngine — generates recommendation explanations
    - Important methods:
        - Food Agent.processRequest()
        - ConstraintExtractor.extractConstraints()
        - RestaurantService.search()
        - MemoryService.retrieveRelevantMemory()
        - Food Agent.rankRestaurants()
        - Food Agent.explainRecommendations()
    - Execution: The agent processes the user's natural-language request, extracts constraints, retrieves restaurants and relevant memories, filters and ranks candidates, and generates explanations. The Facade pattern coordinates the workflow, while the Strategy pattern supports different ranking algorithms.
- F04: Personalized Taste Profile
    - Related Use Case: UC04: Personalized Preference Profile
    - Related Sequence Diagram: SD02: Create Food Review
    - Classes involved:
        - UserPreferences — stores explicit food preferences
        - FoodMemory — stores past experiences
        - MemoryService — retrieves and updates memories
        - TasteProfileUpdater — updates preferences based on feedback
        - FeedbackPublisher — notifies feedback observers
    - Important methods:
        - MemoryService.retrieveRelevantMemory()
        - MemoryService.updateMemory()
        - TasteProfileUpdater.onFeedback()
        - TasteProfileUpdater.updatePreference()
    - Execution: The system collects ratings, reviews, and feedback, then updates the user's preference profile. The Observer pattern allows FeedbackPublisher to notify TasteProfileUpdater when new feedback is available.

- F05: Quick Photo Experience Sharing
    - Related Use Case: UC05: Share Food Photo
    - Related Sequence Diagram: SD02: Create Food Review
    - Classes involved:
        - FoodReview — stores the photo and caption
        - User — creates and publishes the post
        - FoodMemory — preserves relevant experience information
        - MemoryService — saves the experience for future use
    - Important methods:
        - FoodReview.publish()
        - MemoryService.saveMemory()
        - FoodMemory.addReview()
    - Execution: The user selects a photo, optionally adds a restaurant and caption, and publishes the post. The system stores the review and may save its associated experience to personal food memory for future recommendations.
- F06: Friends' Activity Feed
    - Related Use Case: UC06: View Friends' Activity Feed
    - Related Sequence Diagram: SD05: View Friends' Activity Feed
    - Classes involved:
        - User — maintains friends and accesses their feed
        - FriendsFeed — retrieves friends' posts
        - FoodReview — represents each food post
        - Restaurant — provides associated restaurant details
    - Important methods:
        - User.getFriendsFeed()
        - FriendsFeed.getPosts()
        - FriendsFeed.getPostByFriends()
        - Restaurant.getDetail()
    - Execution: When the user opens the feed, the system retrieves posts from friends and displays their photos, captions, and associated restaurants. The user can open a restaurant and save it to their map or collection.
- F07: Restaurant Discovery and Search
    - Related Use Case: UC07: Search and Discover Restaurants
    - Related Sequence Diagram: SD06: Search Restaurants
    - Classes involved:
        - RestaurantService — coordinates restaurant searches
        - RestaurantRepository — retrieves restaurant records
        - Restaurant — represents restaurant information
        - CandidatesFilter — filters matching results

    - Important methods:
        - RestaurantService.search()
        - RestaurantRepository.findByFilters()
        - CandidatesFilter.filter()
        - RestaurantService.getById()
    - Execution: The user enters a search query and optional filters. RestaurantService retrieves matching records through RestaurantRepository, and CandidatesFilter filters the results before they are displayed. This feature uses deterministic search and filtering.
- F08: Restaurant Recommendation History
    - Related Use Case: UC08: View Recommendation History
    - Related Sequence Diagram: SD07: View Recommendation History
    - Classes involved:
        - Food Agent — generates recommendations
        - HistoryRecorder — records recommendation results
        - RestaurantService — retrieves restaurant details
    - Important methods:
        - Food Agent.getRecommendations()
        - HistoryRecorder.recordRecommendations()
        - RestaurantService.getById()
    - Execution: After generating recommendations, the system records them through HistoryRecorder. When the user opens recommendation history, the stored results are displayed so the user can revisit restaurants. Recommendation generation is AI-based, while history storage and retrieval are deterministic.
- F09: Automatic Categories and Tagging
    - Related Use Case: UC09: Generate and Edit Automatic Tags
    - Related Sequence Diagram: SD08: Generate Automatic Categories and Tags
    - Classes involved:
        - ConstraintExtractor — supports tag generation
        - LLMEngine — generates suggested tags
        - Restaurant — stores restaurant tags
        - Tag — represents an individual category or tag
    - Important methods:
        - ConstraintExtractor.generateTags()
        - LLMEngine.generateTags()
    - Execution: The system analyzes available restaurant information and generates relevant tags using the LLM. The tags are associated with the restaurant, and the user can correct or remove inaccurate suggestions.

- F10: Save and Custom Food Collections
    - Related Use Case: UC10 — Manage Food Collections
    - Related Sequence Diagram: SD02 — Save Restaurant to Collection
    - Classes involved:
        - User — initiates restaurant-saving operations
        - Collections — manages collection contents
        - SavedCollection — represents saved restaurants
        - CustomCollection — represents user-defined collections
        - RestaurantRepository — retrieves restaurant records
    - Important methods:
        - User.saveRestaurant()
        - RestaurantRepository.findById()
        - Collections.addRestaurant()
        - Collections.removeRestaurant()
        - Collections.getRestaurants()
    - Execution: When the user saves a restaurant, the system retrieves its record and adds it to the selected collection. Users can later view or remove restaurants from collections and display collection contents on the personal map.